# Cross-Assembler für C8205 / C8205Z v.02

Die gesamte Software läuft sowohl unter Linux als auch Windows 10.
Die Software ist in C (gcc) programmiert, die grafische Oberfläche benutzt node.js

## 1. Installation

Zum Zweck der Installation ist die Datei C8205.zip in einem geeigneten Verzeichnis zu entpacken. Beim Entpacken wird ein Unterverzeichnis angelegt, in dem sich sowohl Windows- als auch Linux-Binaries befinden.

### 1.1 Verzeichnisstruktur

Der Assembler liegt im Unterverzeichnis Assembler des Gesamtprojekts C8205. Es enthält folgende Struktur (Beispielprojekte gekürzt):

```
ls -1R
./Assembler:
c82ass                  # Batch-Datei zum Erzeugen des Assemblers
c82ass.sh               # Ausführen des Assemblers
Dokumentation           # diese Dokumentation
_Linux                  # Linux-Binary des Assemblers
projekdetails           # zuletzt benutzte Aufrufparameter
projekdetails.<Projekt> # gesicherte Aufrufparameter je Projekt
Projekt_1               # Projektverzeichnis (Beispiel, mit Unterverzeichnissen)
Proj_FeldSortieren      # Projektverzeichnis (Beispiel, ohne Unterverzeichnisse)
Sourcecode              # C-Quellcode des Assemblers selbst
_Windows                # Windows-Binary des Assemblers

./Assembler/Dokumentation:
'C8205_Assembler Dokumentation.odt'

./Assembler/_Linux:
c82ass

./Assembler/Projekt_1:
bsp_5.15
bsp_5.16

./Assembler/Projekt_1/bsp_5.16:
bsp_5_16.asm  bsp_5_16.dmp  bsp_5_16.log  bsp_5_16.mqo  bsp_5_16.obj  bsp_5_16.sym
bsp_5_16.test

./Assembler/Proj_FeldSortieren:
code.asm  code_wdhl_bef.asm  code.log  code.mqo  code.obj  code.sym  code.test

./Assembler/Sourcecode:
c8205_op_codes.c  c82_austausch.c  c82_austausch.h  ca82ass.c
ca82_protokoll.c  ca82_protokoll.h  ca82_standards.c  ca82_standards.h
c82ass.makros
```

./Assembler/_Windows:

Projektverzeichnisse können bebiebig benannt sein, auch Unterverzeichnisse enthalten. In Ihnen steht der Quelltext , die Erzeugte Objektdatei (meint die K1K2- ladefähige Textdatei) und weitere Hilfsdateien. Die Programme c82ass, c82load, c82dump, c82simi  erwarten ihre Dateien immer in einem Projektunterverzeichnis, das im gleichen Pfad wie sie selbst liegen muß.
In der vorliegenden Projektstruktur ist nur c82ass enthalten; die Programme c82load, c82dump und c82simi (sowie der C8205Simulator) liegen nicht bei.

c82ass ist ein Shell-Script zum Erzeugen des Assemblers . Die *.bat Datei leisten das gleiche für den MinGW-w64 Crosscompiler unter Windows.
Der Pfad _Linux enthält alle Linux-Binaries und Libraries, der Pfad _Windows die Pendants für Windows 10.
Sourcecode enthält außschließlich allen C-Quellcode.

# 2. **Überblick**

Sowohl die Programme c82ass.sh als auch c82ass.bat Pendant werden über Aufrufparameter versorgt. Bei jedem Start eines der Programme werden die jeweils letztbenutzten Parameter automatisch in der Datei projekdetails abgelegt, so dass ein Folgestart ohne Angabe von Parametern gelingt. Zur Änderung müssen die Parameter einfach mit veränderten Werten wieder angegeben werden.

$c82ass.sh -h
C8205 Assembler C82aas Version 0.2
Usage: c82ass.sh -p Projektverzeichnis -s Quellcode-Datei

# 3. Der Assembler c82ass

*Version: 0.2*

1. Programmstart

Der Assembler **c82ass** erwartet eine Eingabedatei mit <quelltext> unter einem Dateinamen **.asm**.
Er ermittelt im Pass 1 die Literaltabelle, die im Logfile **.log** ausgegeben wird.
Er ermittelt im Pass 2 die Symboltabelle, die ebenfalls im Logfile **.log** ausgegeben wird.
Er erzeugt im Pass 3 eine Datei mit modifiziertem (unformatierten) Quelltext, die im **.mqt** File ausgegeben wird. Hier sind die Literale aufgelöst.
Daraus erzeugt er ebenfalls im Pass 3 die Kennzeichenbefehle für das „Kleine Eingabeprogramm“, die im Objektfile **.obj** ausgegeben werden.
Auf STDOUT gibt der Assembler dabei den formatierten Quelltext (einschießlich der aufgelösten Literale) aus.

Der Aufruf erfolgt zum Beispiel in der Form:

$c82ass.sh -p PzBeispiele_4 -s bsp_5.17_Oktaldruck.asm

Fehlermeldungen erfolgen innerhalb der Ausgabe des formatierten Quelltext auf STDOUT.
Die <adresse> der Befehle im Objektfile beginnen immer bei 0000 bzw. bei dem durch **start** eingestelltem Wert.

Der Assembler erzeugt bearbeitet Dateien im Verzeichnis der *.asm -Datei:
Projekt_1/bsp_5.16/

**bsp_5_16.asm** - Quelltext
bsp_5_16.dmp
**bsp_5_16.log** - Literaltabelle, Symboltabelle, Tabelle der absoluten Symbole
**bsp_5_16.mqo** - aufbereiteter Quelltext für c82load, der ihn mit dem Maschinencode kombiniert
- für c82simi, der ggf. bei der Einzelschrittverarbeitung die Kommentare einblendet
**bsp_5_16.mqt** - modifizierter Quelltext, Literale eingearbeitet
**bsp_5_16.obj** - mit c82load ladefähiger Objektcode K1K2
**bsp_5_16.sym** - Symboldatei , wird von c82dump benutzt, um zum Trommeldump Symbole zuzuordnen
**bsp_5_16.l82** - mit C8205Simulator ladefähiger Objektcode K1K2 über das Pseudo-Gerät Lochstreifenleser
**bsp_5_16.**.test - Trommeldatei im Format des Interpreters (`Adresse: Wert # Quelltextzeile`); Grundlage für Testaufträge, s. `C8205-Interpreter-Durchführung_eines_Tests.md

2. Sprachdefinition

<zahl> ::= <oktalzahl> | <dezimalzahl>
<oktalzahl> ::= oktalziffer==0 [ oktalziffer [ oktalziffer … ]]
<oktalziffer> ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7
<vorzeichen> ::= -
<dezimalziffer> ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9
<dezimalzahl> ::= [<vorzeichen>] dezimalziffer<>0 [ dezimalziffer [ dezimalziffer … ]]
<alphazeichen> ::= abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ
<sonderzeichen> ::= % : + - / ` , . < " [ > ' ~ ] # * ( = ; ) ? !

<wert> ::= <oktalzahl> | <dezimalzahl> | <symbol>
<rechenzeichen> ::= + | - | * | /
<speicherplatz> ::= Speicherstelle auf dem Trommelspeicher, fortlaufend nummeriert durch eine Oktalzahl mit dem Wertebereich 00000 bis 07777.
<symbol> ::= <adresssymbol>|<literalsymbol>| (leer)
<adresssymbol> ::= Zeichenfolge aus 1 bis 32 Zeichen:
<alphazeichen> [ <alphazeichen> | <dezimalziffer> [ … ]],
abgeschlossen durch Doppelpunkt
Sie wird vom Programmierer frei definiert.
<literalsymbol> ::= ‚l‘ (kleines L) gefolgt von einer Oktalziffernfolge von genau 4 Oktalziffern , abgeschlossen durch Doppelpunkt.
Sie wird vom Assembler automatisch aus dem Inhalt des Literalzählers vergeben.
<ausdruck> ::= <werteausdruck> | <literal>
<operand> ::= <werteausdruck> | <literal> | (leer)
<adressausdruck> ::= <wert><rechenzeichen>[<wert><rechenzeichen> … ]
Einschränkungen: kein negatives Vorzeichen
Wertebereich nur von 00000 … 07777
Nur maximal ein Symbol erlaubt.
<werteausdruck> ::= <wert><rechenzeichen>[<wert><rechenzeichen> … ]
<literal> ::= **#**<werteausdruck> | **@**<adressausdruck>
Literal bedeutet, dass der nummerische Wert von <werteausdruck> oder <adressausdruck> auf einem vom Assembler selbst festgelegten <speicherplatz> gespeichert wird. Diesem Speicherplatz wird vom Assembler ein <literalsymbol> zugeordnet.
<adresse> ::= nummerischer Wert eines <symbol>, der einen <speicherplatz> adressiert.
<kommentar> ::= **;** (Semikolon) gefolgt von ASCII-Zeichen. | (leer)
Innerhalb der \-Zeichen von **dt** wird ; nicht als Kommentar betrachtet.
<trenner> ::= Tabulator[<trenner>[...]] | Leertaste<trenner>[...]]
<befehl> ::= [<symbol>] <trenner><operationscode> [<trenner><operand>]
[<trenner><kommentar>]
Alle Befehle müssen auf einer Zeile stehen.

<operationscode> ::= <maschinenbefehl> | <assembleranweisung>
<assembleranweisung>::=start | end | db | dw | da | dt | dp | dz | equ
<maschinenbefehl> ::= <oktalziffer><oktalziffer><oktalziffer>

<quelltext> ::=<befehl>[<befehl> [ … ]]

## 3. Assembleranweisungen

### 3.1 Definiere Bereich

[<symbol>] **db** <zahl>

Hält <zahl> Speicherplätze frei, ohne ihren Inhalt zu definieren

### 3.2 Definiere Bereich auf Null

[<symbol>] **dbn** <zahl>

Hält <zahl> Speicherplätze frei, setzt ihren Inhalt auf 0.

### 3.3 Definiere Wert

[<symbol>] **dw** <werteausdruck>

- Speichert den nummerischen Wert von <werteausdruck> auf dem <speicherplatz>
- Es wird keine Hauptleitadresse addiert.

[<symbol>] **dw** <literal>

- Erzeugt am dem Programmende automatisch aus <literal> eine Definition der Art:
  <literalsymbol> DW <werteausdruck>.
- Speichert die <adresse> **von** <literalsymbol> auf dem aktuellen <speicherplatz>.
- Es wird keine Hauptleitadresse addiert.

### 3.4 Definiere Adresse

[<symbol>] **da** <adressausdruck>

Speichert den nummerischen Wert von <adressausdruck> auf dem <speicherplatz> verschoben um «9 in den Adreßstellen.
Zum <adressausdruck> wird die 1. Hauptleitadresse addiert.

[<symbol>] **da** <literal>

- Erzeugt am dem Programmende automatisch aus <literal> eine Definition der Art:
  <literalsymbol> da <adressausdruck>.
- Zum <adressausdruck> wird die 1. Hauptleitadresse addiert.
- Nimmt stattdessen die <adresse> von <literalsymbol> in den befehl, der das <literal> enthält..

### 3.5 Definiere Text zur Ausgabe von Zeichens

[<symbol>] **dt**[**o**|**m**] \<alphazeichen> | <leerzeichen> | <dezimalziffer>
<sonderzeichen> [ <alphazeichen> | <leerzeichen> |
<dezimalziffer> | <sonderzeichen>[...]]\

- gespeichert wird ein Code gemäß R300 – Code Ausgabeadresse C8205 (siehe R300-code.pdf). Die Abspeicherung erfolgt als Folge von Speicherplatzinhalten in der Form (oktal): 0000 xxxx 030, wobei xxxx die Ausgabeadresse ist.
  Die Zeichen „\“ am Anfang und am Ende sind nur erforderlich, wenn der Text Leerzeichen enthält.
  Außer für das erste Zeichen werden die Umschaltzeichen automatisch aus dem Text abgeleitet. Achtung: Jedes neue **dt** beginnt wieder mit der Annahme, daß die Einstellung KB/GB zum ersten Zeichen des Textes paßt.

  **dto** verhält sich wie **dt**, außer dass generell KEINE Umschaltzeichen *automatisch* eingefügt werden. Das bedeutet, in einer dto-Anweisung kann man KEINE gemischte Groß- und Kleinschreibung definieren ! (auch keine Zahlen und Großbuchstaben usw.)
  **dtm** verhält sich wie **dt**, außer dass vor dem ersten Zeichen generell mit Umschaltzeichen, dem ersten Buchstaben entsprechend, automatisch eingefügt wird.

3.6 Definiere Zeichen (spezielle Sonderzeichen)

[<symbol>] **dz** $wrzl | $sm | $inkb | $inkb | $irrz | $irrs | $trans | $bm | $vorl | $irrb | $wr | $tab | $zr | % | ; | + | ` | / | , | . | -

- gespeichert wird ein Code gemäß R300 – Code Ausgabeadresse C8205 (siehe Tabelle 2). Die Abspeicherung erfolgt auf einem Speicherplatz in der Form (oktal): 0000 ssss 030. Eine automatisch Ergänzung von Umschaltzeichen wie bei **dt** erfolgt hier generell NICHT.

Findet man beispielsweise **dz $wrzl** zu kompliziert, kann man das mit einem Makro vereinfachen:

```
WRZL:   MACRO
            dz      $wrzl  ; Zeilenschaltung
        MEND
```

3.7 Beginn des Quellcodes

**start** [<zahl> ]

- Nur als 1. Anweisung im <quelltext> zulässig.
  Eine Maschinencodeausgabe erfolgt nicht.
  Ist <zahl> angegeben, wird die 1. Leitadresse in den „Lochstreifen“ ausgegeben.
  Es wird dann durch c82load auf diese 1.Leitadresse + Ladeadresse geladen.
  Ladeadresse existiert dabei nur bei Laden mit c82load.

3.8 Definition Relativadresse

[<symbol>] **equ** <werteausdruck> | *

- Definiert für das <symbol> einen <werteausdruck>.
  Der Befehlszähler wird auf diesen Wert gesetzt.
  Ist ‚*‘ angegeben, wird dem Symbol dagegen der aktuelle Befehlszählerstand zugewiesen.
  Eine Maschinencodeausgabe erfolgt nicht.
  Im Objektcode für das „Kleinen Eingabeprogramm“ wird der Kenzeichenbefehl für das Setzen der Relativadresse ausgegeben (KZ=20).
  Das weitere Laden erfolgt auf 1.Leitadresse + Ladeadresse + Relativadresse.
  Ladeadresse existiert dabei nur bei Laden mit c82load.
  Die Anweisungen **equ100**, **equ40** und **equ10** runden entsprechend auf die nächsten volle Speieradresse auf.

3.9 Definition Absolutadresse

[<symbol>] **abs** <oktalzahl> | <dezimalzahl>

- Definiert für das <symbol> eine absolute <adresse>.
  Eine Objektcodeausgabe erfolgt nicht.
  Eine Maschinencodeausgabe erfolgt nicht.
  Beachte: Symbole, die durch **abs** definiert wurden, sind nicht verschieblich. Anweisungen, in denen ein solches Symbol im Adreßausdruck oder Werteausdruck vorkommt, werden mit K1K2=00 generiert !
  **abs** kann mehrfach mit dem gleichen Wert definiert werden, ohne Fehler zu erzeugen, aber nicht mehrfach mit dem gleichen Symbol für unterschiedliche Werte.

3.10 Ende des verschieblichen Teils des Quelltextes

[<symbol>] **end**

- Hinter dieser Anweisung sollten nur noch **abs** oder **equ**-Anweisungen folgen.
  Der Assembler ordnet hinter „**end**“ die automatisch erzeugten Literaldefinitionen ein, so dass sie noch zum verschieblichen Teil hinzukommen.

Ein nachfolgendes **equ** käme also HINTER die Literale.
Eine Maschinencodeausgabe erfolgt für „**end**“ nicht.

Tabelle 2: Sonderzeichen für **dz** – Anweisung

Spalte 1: String, der als Operand bei **dz** anzugeben ist
Spalte 2: 4 Oktalziffern, die die 8 Bit des 030 Ausgabecodes enthalten

| | |
|---|---|
| $wrzl | 02720 |
| $sm | 02720 |
| $inkb | 01170 |
| $ingb | 0570 |
| $irrz | 03770 |
| $irrs | 0160 |
| $trans | 01770 |
| $bm | 03520 |
| $vorl | 0 |
| $irrb | 0760 |
| $zr | 0320 |
| $tab | 01550 |
| $wr | 01120 |
| % | 01740 |
| ; | 0150 |
| + | 0400 |
| ` | 01600 |
| / | 01410 |
| , | 01530 |
| . | 0730 |
| - | 01000 |

Tabelle 3: Ausgabe-Umschlüsselung außer Sonderzeichen dz

Spalte 1: ascii -Kleinbuchstaben
Spalte 2: ascii -Großbuchstaben
Spalte 3: 4 Oktalziffern, die die 8 Bit des 030 Ausgabecodes enthalten
Spalte 4: ascii -code der Großbuchstaben
Spalte 5: ascii -code der Kleinbuchstaben

| | | | | |
|---|---|---|---|---|
| 1 | < | 0010 | 60 | 49 |
| 2 | " | 0020 | 34 | 50 |
| 3 | ≈ | 0230 | 0 | 51 |
| 4 | [ | 0040 | 91 | 52 |
| 5 | > | 0250 | 62 | 53 |
| 6 | ' | 0260 | 39 | 54 |
| 7 | ≋ | 0070 | 0 | 55 |
| 8 | ~ | 0100 | 126 | 56 |
| 9 | ] | 0310 | 93 | 57 |
| 0 | # | 0200 | 35 | 48 |
| % | * | 1740 | 42 | 37 |
| Tabulator | Tabulator | 1550 | 9 | 9 |
| q | Q | 1300 | 81 | 113 |
| w | W | 1660 | 87 | 119 |
| e | E | 0450 | 69 | 101 |
| r | R | 1110 | 82 | 114 |
| t | T | 1630 | 84 | 116 |
| z | Z | 1710 | 90 | 122 |
| u | U | 1440 | 85 | 117 |
| I | I | 0510 | 73 | 105 |
| o | O | 1060 | 79 | 111 |
| p | P | 1270 | 80 | 112 |
| : | ( | 0150 | 40 | 58 |
| + | = | 0400 | 61 | 43 |
| a | A | 0610 | 65 | 97 |
| s | S | 1420 | 83 | 115 |
| d | D | 0640 | 68 | 100 |
| f | F | 0460 | 70 | 102 |
| g | G | 0670 | 71 | 103 |
| h | H | 0700 | 72 | 104 |
| j | J | 1210 | 74 | 106 |
| k | K | 1220 | 75 | 107 |
| l | L | 1030 | 76 | 108 |
| ` | ; | 1600 | 59 | 96 |
| / | ) | 1410 | 41 | 47 |
| SatzMarke | WRZL | 2720 | 10 | 10 |
| y | Y | 1500 | 89 | 121 |
| x | X | 1470 | 88 | 120 |
| c | C | 0430 | 67 | 99 |
| v | V | 1650 | 86 | 118 |
| b | B | 0620 | 66 | 98 |
| n | N | 1050 | 78 | 110 |
| m | M | 1240 | 77 | 109 |
| , | ? | 1530 | 63 | 44 |
| . | ! | 0730 | 33 | 46 |
| - | | 1000 | | 45 |
| Leertaste | ZwR | 0320 | 32 | 32 |

# 4. Der Makro-Präprozessor des Assemblers

Der Päprozessor ist sehr einfach gehalten und hat eine starre Syntax. Er prüft, ob eine Operation ein Makroname ist. Die Prüfung erfolgt in der Textdatei ca82ass.makros, die erweitert werden darf.

Jeder der dort definierten Makros beginn mit dem Makronamen unmittelbar gefolgt von einem Doppelpunkt und nach einem Tab das Schlüsselwort MACRO.
Müssen dem Makro Parameter übergen werden, folgt nach MACRO, getrennt durch Leerzeichen, die Anzahl der Parameter. Es sind Maximal 9 Parameter zulässig.
Danach kommen die Definitionsanweisungen des Makros. Sie kann die formalen Parameter **$**x mit x=1..9 enthalten, die durch die Übergabeparameter ggf. mehrfach ersetzt werden.
Jeder Makro wird durch das Schlüsselwort <tab> MEND abgeschlossen.

Im Definitionsteil kann für jeden Parameter eine Prüfliste angegeben werden. Ist sie angegeben, darf der betreffende Parameter nur die Werte der Liste annehmen. Eine Prüfliste wird durch $Prx mit x=1..9 definiert. Sie muß auf einer Zeile für sich stehen.

Die Definition der zu generierenden Anweisungen kann duch eine $IF-$ELIF-$ENDIF Struktur gesteuert werden. Sie kann mehrfach auftreten, darf aber nicht geschachtelt werden. Die Schlüsselworte $IF $ELIF $ENDIF müssen jeweils auf einer Zeile für sich stehen.
Hinter $IF und $ELIF muß mindestens einer, können bis zu 4 Bedingungsblöcke angegeben werden. Ein Bedingungsblock besteht aus 3 Strings: <wert1> <vergleichszeichen> <wert2>.
Als <vergleichszeichen> sind: >, <, =, >=, <= zugelassen. Jeder folgende Bedingungsblöcke ist durch **|** (für logisch ODER) oder **&** (für logisch UND) zu verknüpfen.
Achtung: Die Bedinungsblöcke habe eine IMPLIZITE Klammerung der Art:
**(((** <bedingungsblock1> [|&] <bedinungsblock2> **)** [|&] <bedinungsblock3> **)** [|&] <bedinungsblock4> **)**

Innerhalb eines Definitionsblockes sind Kommentare zugelassen. **#** leitet einen Kommentar ein, der nur in der Makrodefinition gilt. **#** muß auf einer Zeile für sich stehen.
**;** leitet einen Kommentar ein, der auch generiert wird. Es kann auf einer Ziele für sich oder in der Anweisung vorkommen.

**Beispiel**:

```
ANWAHL:  MACRO 1
 $PR1 4000 4100 4200 4300 4400 4500 4600 4700 K0 K1 K2 K3 LSL1 LSL2 LKL1 LKS1 LSS1 SW1I SW1O SW2O KONI
 # Generieren einer Anwahl für Zeichenkanäle
    $IF  $1  =  4000 | $1 = K7
      030 4000  ; 4000 oder K7
    $ELIF  $1  =  4100 | $1 = K6
      030 4100  ; $100 oder K6
    $ELIF  $1  =  4200 | $1 = K5
      030 4200  ; 4200 oder K5
    $ELIF  $1  =  4300 | $1 = K4 | $1 = LKS1
      030 4300  ; 4300 oder K4 oder LKS1
    $ELIF  $1  =  4400 | $1 = K3 | $1 = LKL1 | $1 = SW2O
      030 4400  ; 4400 oder K3 oder LKL1 oder SW2 Output
    $ELIF  $1  =  4500 | $1 = K2 | $1 = LSL2 | $1 = SW1O
      030 4500   ; 4500 oder K2 oder LSL2 oder SW1 Output
    $ELIF  $1  =  4600 | $1 = K1 | $1 = LSL1 | $1 = LSS1
      030 4600  ; 4600 oder K1 oder LSL1 oder LSS1
    $ELIF  $1  =  4700 | $1 = K0 | $1 = SW1I | $1 = KONI
      030 4700  ; 4700 oder K0 oder SW1 Input oder KON Input
    $ENDIF
      MEND
```

Aufruf: **ANWAHL LSS1** erzeugt: 030 04600 ; 4600 oder K1 oder LSL1 oder LSS1

Der Macroaufruf im Assemblerprogramm darf ein Label haben:

```
input_loop:     ANWAHL   LSL1
                …
```

In einem solchen Fall wird generiert:

```
;   ANWAHL   LSL1
input_loop:     030      04600
```

Enthält der Makro z.B. selbst eine Schleife mit einem Label:

```
BSP1:           MACRO
# Makro enthält eine Schleife
start%:         211      …
                …
                000      start%
                MEND
```

So wird bei einem Macroaufruf:

```
loop:           BSP1
```

folgende Anweisungsfolge generiert, wenn die erste zu generierende anweisung ein eigenes Label hat:

```
loop            equ      *
start0001       211      …
                …
                000      start0001
```

Hat die erste Anweisung kein eigenes Label, entfällt die Equ-Anweisung und die erste Anweisung erhält das Label des Makroaufrufes.

Ein Macro SOLLTE kein „normales“ Label der Assemblersprache enthalten, sonst kann er nur einmalig aufgerufen werden. Stattdessen sollte ein generisches Label verwendet werden, das mit einem Buchstaben beginnt, ggf. weitere Buchstaben und Zahlen enthält, und vor dem Doppelpunkt ein %-Zeichen hat. Dieses %-Zeichen wird durch die vierstellige Macroaufrufnummer ersetzt.

Makro Aufrufparameter, die exakt 4 Ziffern enthalten, und nur diese (!!) , werden ggf. als 4-Ziffern-String geprüft (durch $PRx), sie erhalten bei der Benutzung (Ersetzung des formalen Parameters) automatisch eine Vornull.

```
ANWAHL  4600
```

erzeugt:

```
;    ANWAHL 4600
          030      04600
```

# 5. Compilierungsbeispiel

c82ass.sh -p Projekt_1 -s bsp_5_16

## 5.1. Quelltext bsp_5_16.asm und Assemblerlauf

```
C8205 Assembler C82aas Version 0.2
Projektverzeichnis: Projekt_1
Benutze für -s die Datei: bsp_5_16.asm
Eingabedatei: Projekt_1/bsp_5.16/bsp_5_16.asm
Modifizierte Datei: Projekt_1/bsp_5.16/bsp_5_16.mqt
Log-Datei: Projekt_1/bsp_5.16/bsp_5_16.log
Maschinencode-Datei: Projekt_1/bsp_5.16/bsp_5_16.obj
Hilfsdatei: Projekt_1/bsp_5.16/bsp_5_16.mqo
Symboltabelle-Datei: Projekt_1/bsp_5.16/bsp_5_16.sym
Verarbeite Quellcode: Projekt_1/bsp_5.16/bsp_5_16.asm

Pass 1
Verarbeite Projekt_1/bsp_5.16/bsp_5_16.asm, Ermittle Literale

Pass 2
Verarbeite Projekt_1/bsp_5.16/bsp_5_16.mqt, Löse Symbole auf

Pass 3
Verarbeite Projekt_1/bsp_5.16/bsp_5_16.mqt, Erzeuge Kennzeichen-Befehle.
; C8205 Heft 2 Programmierung in Maschinensprache
; Beispiel: 5.16 Umspeichern eines Vektors
;
                        start
                        000     beginn
aqv:                    da      qvektor    ; Adrese der ersten Komponente Quellvektor
dqv:                    dw      05000    ; Dimension von qvektor gezählz in Adreßeinheiten
azv:                    da      zvektor    ; Adresse der ersten Komponente Zielvektor
adreinheit:             dw      01000    ; Adresseinheit
spbef:                  dw      0240    ; Speicherbefehl
i:                      db      1    ; Arbeitszelle für i
beginn:                 241     i    ; I:=0
schleife:               210     azv    ; ac:=(azv+i)
                        210     spbef    ; ac:=(azv+i)240
                        240     genbef    ; Arbeitszelle genbef:= (qzv+i) 240
                        211     i    ; ac:=<i>
                        210     aqv    ; ac:=qvektor+i
;  251: Laden des Inhaltes einer Speicherstelle, deren Adresse im AC steht in den AC.
                        251     0    ; ac:=<qvektor+i>
genbef:                 db      1    ; hier steht der Befehl 240 (qzv+i)  Speichern
                        211     i    ; ac:=<i>
                        210     adreinheit    ; ac:=i+1
                        240     i    ; i:=ac
                        330     dqv    ; vergleicher 3, falls i == <dqv>
                        170     halt    ; sprung bei gleich
                        000     schleife
halt:                   020     0    ;halt
qvektor:                dw      1
                        dw      2
                        dw      3
                        dw      4
                        dw      5
                        equ     0100
zvektor:                db      5
ende:                   end


Ausgabe als Minimalprogramm-Objekt in: Projekt_1/bsp_5.16/bsp_5_16.obj
```

5.2. Durch c82ass erzeugter Objektcode (K1K2-Programmcode) für „Kleines Eingabeprogramm“

```
0000: 01 0007 000.
0001: 01 26000.
0002: 00 00000005000.
0003: 01 100000.
0004: 00 00000001000.
0005: 00 00000000240.
0006: 20 0007.
0007: 01 0006 241.
0010: 01 0003 210.
0011: 01 0005 210.
0012: 01 0016 240.
0013: 01 0006 211.
0014: 01 0001 210.
0015: 00 0000 251.
0016: 20 0017.
0017: 01 0006 211.
0020: 01 0004 210.
0021: 01 0006 240.
0022: 01 0002 330.
0023: 01 0025 170.
0024: 01 0010 000.
0025: 01 0000 020.
0026: 00 00000000001.
0027: 00 00000000002.
0030: 00 00000000003.
0031: 00 00000000004.
0032: 00 00000000005.
      20 0000 0100.
0100: 20 0105.
/
```